First Quarter Design Project Fall 2024

Project Design Specification and Evaluation Sheet

Student Name: Arjanveer Samra

Project Name: Environmental Monitoring Automation System

Project Goal: What is the overall goal or function of the project? What particular customer need does it provide? What problem does it solve or what important capability does it provide?

The general goal of this project is to create an environmental monitoring system. This system will be able to read temperature, humidity, and light intensity, and detect motion. Overall, the project addresses the need for accessible and easy-to-use monitoring devices that can be used in homes, offices, and other buildings. All the environmental data is nicely displayed on the LCD making it easier to read. For our motion detection and light sensor, however, we included a special capability. We have LEDs that help give you the data. The light intensity can be out of 0-100%(as displayed on the LCD), and the RGB LED light changes depending on the level of intensity. From 0-39% the light turns red, from 39-59% the light turns blue, and from 60-100% it turns green, this lets you know visually if the light is low, medium, or high. For motion detection, if the PIR motion sensor detects motion, the yellow LED will activate for two seconds. This meets customers' needs as it allows them to read the temperature and humidity of any given area or room, judge the ambient lighting in any space, and detect any motion going on alerting the customer immediately.

Intended Customer / User: Who is most likely to want to use your project? Any specific age groups, or geographic locations, or particular occupations who might be more likely to use it?

The project is made for individuals, groups, or organizations that need a simple and effective way of monitoring environmental conditions. Some of our common demographic would be homeowners and renters that need these temp, humid, and light sensors, as well as motion detection to better help with security. Furthermore, another group would be small business owners looking to open up new buildings or offices, since those new places are going to be heavily upgraded with our environmental sensors. Literally, anyone interested in the creation of a building could be a potential customer for us. The occupations that are more likely to use our product would be engineers, construction workers, and environmental enthusiasts since they would relate to our product the most and will need it more based on their endeavors. There is no specific geographical location as this could be used across the world at all different types of conditions. In addition, it would be made available for those who are eighteen years of age or older. People who can accept ownership of a property, and use our product effectively to meet their needs.

Product Use Environment: Will your project be used indoors or outdoors; in a home, office, factory or other setting?

The product is made for primarily indoor use. Places like homes, offices, classrooms, laboratories, factories, and other indoor places. But it can be adjusted for outside use as long as there aren't extreme temperature or humidity conditions.

Temperature Range: **32°F – 122 °F or 0°C – 50 °C**

Market Research: What alternatives are currently available to a potential customer to meet this need, and what do they cost?

Well, right now there are plenty of environmental monitoring automation systems out there that can detect motion and a variety of other things like light, humidity, and temperature.

Some examples of Smart Home Monitoring systems that are already out their and their costs are Net Protect by Google, it includes motion, temperature, and smart alert safety all for the price of about $120-$150 per unit.

Ecobee Smart Sensors is another company that can detect temperature, humidity, and motion, and they sell anywhere from $80-$100 for every two packs of sensors.

Airthings Wave Plus is one company that offers a standalone environmental monitor that monitors indoor air quality, temperature, and humidity features that come along with built-in radon detection. A product like this is expected to cost about $200.

A much more budget-friendly alternative is the Govee WiFi Thermo-Hygrometer, a $50 temperature and humidity measurer that connects to an app to track the data.

Another popular choice is the Philips Hue Motion Sensor which controls light based on the environment(very similar to my project). It is about $40-$50 and this system caters to those looking for a simple motion-controlled light system.

Lastly, are the Arduino kits that we used, which can cost anywhere from $40-$80, where people can make their own motion or light sensors.

Customer Needs: Features and Functions - What capabilities should the project include to fully meet the customer's needs?

To satisfy the customer's needs fully, several capabilities need to be developed within the project. First, it shall give precise environmental monitoring through sensing temperature, humidity, and light levels, hence allowing the user to track their ambient effectively. This is extremely important for indoor spaces to guarantee comfort and energy efficiency. Motion detection shall also be done, enabling users to have warnings of movements within the space monitored, which improves security. The system should also be provided with a clear, easy-to-read LCD display to indicate current data, hence user-friendly for all ages. The project must also include custom LED lighting that changes color with light intensity for both aesthetic and feedback purposes. It should be reliable and responsive, hence working seamlessly with little delay. With these features, the project will meet the practical and aesthetic needs of its targeted users.

‘

Performance Requirements? – What quantifiable performance specifications must it meet?

Specification	Requirement	Units
Light Intensity should be displayed and change RGB Light Accordingly	The light intensity displayed on the LCD from 0-100%, when it is 0-39% the RGB light turns red, when it is 39-59% it turns blue, and when it is 60-100% it turns green	Using map command to take the given value of the photoresistor and make it from 0 to 100 percentages. From 0-39% Red RGB on, then the blue one for 39-59%, and green for 60-100%.
The LCD display clearly displays all information neatly and nicely in an easy-to-read manner	The LCD display should display temperature and humidity top and bottom then every 5 seconds switch the display to light at the top and motion is detected or not at the bottom, the switch should be smooth and simultaneous. Also, the spacing and the right symbols between degrees and percentages should be used. “Temp: xyzC” “Humidity: xyz%” (5 seconds switch at the same time)	5 seconds the display switches, also 5k milliseconds

	“Light: xyz%” “No Motion/Motion Detected”	
Motion Detected, when the Motion is detected the Yellow LED should light up indicating that motion is detected	So when the motion is detected in front of the PIR motion sensor, the LCD display will read “Motion Detected” and the yellow LED should light up for 2 seconds to indicate that	Yellow LED turns on for two seconds when Motion is detected and displays it on the LCD

Project Learning Requirements Checklist: Design your project to include all learning requirements.

Project Learning Requirements	**Describe what you did for this requirement**
1. Use a library (need functions someone else already wrote for you)	**The project I made utilizes two libraries. One of the libraries is “<LiquidCrystal.h>” and this is to control the LCD display screen. The other library was “<DHT.h>” and this is used to control the DHT 11 sensor and allow it to read and give us the temperature and humidity readings.**
2. Use lots of different programming structures – Describe what you did	**The code I included uses lots of programming structures like if an else statements to change the RGB lights according to the light intensity percentage. I also have a lot of variables that are initialized at the top and used throughout my code. Like “photoresistorRaw”, “smoothedLightLevel”, and “motionDetected”, all these store sensor readings and control the general behavior of my system. They're also all used throughout my code. I also have an interrupt for my PIR motion sensor**

	which is another programming structure. I also have the "updateRGBLED()" which is another variable/programming structure that helps update the RGB according to the light level
3. Use a hardware interrupt feature	**The interrupt is included in the line "attachInterrupt(digitalPinToInterrupt(PIR_PIN), motionDetectedISR, RISING);". As you can tell by the name this interrupt is for the PIR motion sensor. This interrupt is used to immediately detect motion through the PIR sensor. It responds according to the sensor without continuously checking the sensor within the main loop of the program. So in general this interrupt makes it so that the microcontroller in the PIR motion sensor can react to motion instantly, which is essential for detecting motion as soon as it occurs. So making an interrupt for the PIR motion sensor makes it way more effective and functional.**
4a. Use at least one sensor as an input	**This project uses multiple sensor inputs. For instance, the photoresistor which is a light sensor to detect surrounding light levels, a DHT 11 sensor to measure surrounding temperature and humidity, and even a PIR motion sensor, that detects nearby motion.**
4b. Control something that is variable (like motor speed, or brightness)	**This project uses a RGB light to control the color and brightness by the light level detected by the photoresistor. The "updatedRGBLED()" variable makes it so that the intensity of the light makes the LED change color from Red(low 0-39%), Blue(Medium 49-59%), and Green(High 60-100%). The brightness of these colors also changes based on the percentages of their ranges. The variable brightness is controlled by how much light the photoresistor(light sensor) reads.**
4c. Control something that is ON/OFF	**The Motion LED is controlled based on Motion Detection and can be controlled on an off. When the PIR motion sensor detects motion the LED is turned on immediately through the interrupt, the LEd stays**

	on for two seconds, and then after two seconds it is forced to TURN OFF, and has a little pause, before motion can be detected again to turn on the LED.
5. **BONUS FEATURE:** Includes some wireless or remote control feature.	**One wireless feature is the RGB light. As it changes color on it's own depending upon the light levels.**
6. Incorporates a display (like the LCD text display) or sends info back to the computer using the serial connection (USB cable)	**The project does use and integrate the LCD display. It shows real-time info on the data that the project senses. The display reads temperature and humidity and displays it. Then every 5 seconds it switches and displays the light level out of 100 percent and whether motion is detected or not. This allows feedback to the user about the environmental conditions.**
7. Integrates at least one device, sensor, or command that was not included in class projects	**I integrated the photoresistor(light sensor), which we did not include in our projects. I also included the PIR motion sensor, as well as the RGB light, another device and sensor which was not included in any of the projects that we did.**
7a. What is it?:	**It is the photoresistor, RGB light, and PIR motion sensor. Also how the photoresistor (light sensor) is coded along with the RGB light to change its color according to the light intensity that it reads.**
8. I promise to write well-documented, easy-to-read Arduino code. Tell me what you did.	**The Arduino code is structured in a way so its easy to read and follow. Variables, such as "temperature" and "humidity", store the readings from the "DHT11 sensor", while "photoresistorRaw" and "smoothedLightLevel" store data from the "photoresistor". These are updated every loop and drive the system's actions. In terms of detecting motion, the "PIR sensor" triggers an interrupt upon detection. The "motionDetectedISR()" interrupt service routine will activate the motion LED and raise the flag "motionDetected" to true. After 2 seconds, the light automatically turns off. Then for the light sensor the photoresist reads the light level from the**

	"photoresistor" by calling "analogRead(PHOTO_PIN)." Then, "smoothLightLevel()" stabilizes the readings and makes it out of 0-100%, and "updateRGBLED()" updates the RGB LED's color according to light intensity. The colors are red, blue, and green for low, medium, and high light levels, respectively. For temperature and humidity, values read by the "DHT11 sensor" are displayed on the "LCD screen." Every 5 seconds, the display toggles between showing temperature and humidity or light level and motion detection status using the "display state" flag. The "LCD.clear()" method clears the screen each time new data is shown. The code was organized at the end of the project to make the code very easy to follow and change. I also made sure to add a lot of comments explaining what everything does.

Estimated Development Costs:

Prototype Components Costs (itemized list with costs)
Arduino Mega 2560 Board - $35
LCD 16x2 Display - $6
Breadboard - $5
DHT11 Temp and Humidity Sensor - $5
PIR Motion Sensor - $3
Photoresistor(Light Sensor/Photocell) - $1
RGB LED - $2
LED (Motion Detection Light) - $1
Resistors(buy in a set) - $1

Jump Wires(buy in a set) - $3

Potentiometer(10K(need it for LCD Display) - $1

Total cost: $63

.

Estimated # of Hours you will spent working on this project: 18 hours I spent working on this project

User Interface:

Inputs:

PIR Motion Sensor - Detects motion and displays on LCD as well as turning on the motion LED.

Photoresistor - Measures the lighting and is a light sensor, gets a value, the code converts the value from 0-100%, this changes the color of the RGB light depending upon the percentage, 0-39%(Red), 39-59% (Blue), 60-100%(Green), it also displays this percentage on the LCD.

DHT 11 Sensor - Measures the temperature in Celsius and humidity in percentage, displaying it on the LCD.

Outputs:

RGB LED - The RGB LED color changes based on the light level detected by the photoresistor, red is low, medium is blue, green is high, and the percentages to those are seen above for the “inputs” under the “photoresistor”.

LCD Display - Displays the temperature in Celsius at the top line, and humidity as a percentage at the bottom. Every 5 seconds it switches to display Light intensity at the top as a percentage and whether motion is detected or not at the bottom.

Motion LED (normal LED)- Every time motion is detected through the PIR motion sensor, this yellow LED turns on for two seconds

Append a copy of your Code Here;

FinalProject.ino

```ino
#include <LiquidCrystal.h>   // Include LCD library to interact with the LCD screen
#include <DHT.h>             // Include DHT sensor library to read temperature and humidity

// Pin Definitions
#define DHT_PIN 2            // DHT11 data pin connected to digital pin 2
#define PIR_PIN 3            // PIR sensor connected to digital pin 3 (interrupt)
#define PHOTO_PIN A0         // Photoresistor connected to analog pin A0
#define RGB_RED_PIN 6        // RGB LED red connected to digital pin 6 (PWM)
#define RGB_GREEN_PIN 7      // RGB LED green connected to digital pin 7 (PWM)
#define RGB_BLUE_PIN 8       // RGB LED blue connected to digital pin 8 (PWM)
#define MOTION_LED_PIN 4     // Motion LED connected to digital pin 4

// LCD Pins (D4-D7 connected to 11, 12, 13, 5)
LiquidCrystal lcd(9, 10, 11, 12, 13, 5); // (RS, EN, D4, D5, D6, D7)

// Initialize the DHT sensor
DHT dht(DHT_PIN, DHT11);

// Sensor readings and states
int photoresistorRaw; // Raw value from the photoresistor (light sensor)
int smoothedLightLevel; // Smoothed light level value for stable readings
bool motionDetected = false; // State of motion detection (on/off)
float temperature, humidity; // Variables to store temperature and humidity readings

// Variables for timing and display switching
unsigned long lastSwitchTime = 0; // Last time the display was switched
const int displayInterval = 5000; // 5 seconds between display switches
bool displayState = true; // Flag to toggle between displaying temp/humidity or light/motion
unsigned long motionDetectedTimestamp = 0; // Timestamp for when motion was detected

void setup() {
    // Initialize the LCD screen
  lcd.begin(16, 2);  // 16 columns and 2 rows on the LCD screen
  lcd.print("System Ready"); // Display a startup message

  // Initialize the DHT sensor
  dht.begin();

  // Set PIR pin as input for interrupt (motion detection)
  pinMode(PIR_PIN, INPUT);

  // Set PWM pins for RGB LED
  pinMode(RGB_RED_PIN, OUTPUT);
  pinMode(RGB_GREEN_PIN, OUTPUT);
  pinMode(RGB_BLUE_PIN, OUTPUT);

  // Set the motion LED pin as output
  pinMode(MOTION_LED_PIN, OUTPUT);

  // Setup PIR sensor interrupt (triggered on rising edge, meaning when motion is detected)
  attachInterrupt(digitalPinToInterrupt(PIR_PIN), motionDetectedISR, RISING);
}
```

```
void loop() {
  // Read and smooth the photoresistor value to get stable light level
  photoresistorRaw = analogRead(PHOTO_PIN);
  smoothLightLevel(photoresistorRaw);

  // Calculate the light level percentage
  int lightLevelPercent = map(smoothedLightLevel, 50, 900, 0, 100);
  lightLevelPercent = constrain(lightLevelPercent, 0, 100); // Constrain value to range 0-100%

  // Update RGB LED color based on light level
  updateRGBLED(lightLevelPercent);

  // Read temperature and humidity from the DHT sensor
  temperature = dht.readTemperature();
  humidity = dht.readHumidity();

  // Switch between display modes every 5 seconds
  if (millis() - lastSwitchTime > displayInterval) {
    lastSwitchTime = millis();
    displayState = !displayState; // Toggle between displaying temp/humidity or light/motion
  }

  // Display data on the LCD screen
  lcd.clear();
  if (displayState) {
    // Display temperature and humidity
    lcd.setCursor(0, 0);
    lcd.print("Temp: " + String(temperature, 1) + "C");
    lcd.setCursor(0, 1);
    lcd.print("Humidity: " + String(humidity, 1) + "%");
  } else {
    // Display light level and motion status
```

```
    lcd.setCursor(0, 0);
    lcd.print("Light: " + String(lightLevelPercent) + "%");
    lcd.setCursor(0, 1);
    lcd.print(motionDetected ? "Motion Detected" : "No Motion");
  }

  // Keep the LED on for 2 seconds after motion is detected
  if (motionDetected && (millis() - motionDetectedTimestamp >= 2000)) {
    digitalWrite(MOTION_LED_PIN, LOW);  // Turn off LED after 2 seconds
    motionDetected = false;  // Reset motionDetected flag
  }

  delay(500); // Delay for stability to prevent rapid changes
}

// Function to smooth the light sensor readings
void smoothLightLevel(int newValue) {
  smoothedLightLevel = (smoothedLightLevel * 0.9) + (newValue * 0.1); // Weighted average for smoothing
}

// Function to update RGB LED color based on light level
void updateRGBLED(int lightLevel) {
  int redValue = 0, greenValue = 0, blueValue = 0;

  if (lightLevel <= 39) {
    // Red for 0-39%
    redValue = 255;
    greenValue = 0;
    blueValue = 0;
  } else if (lightLevel >= 40 && lightLevel <= 60) {
    // Blue for 40-60%
    redValue = 0;
```

```
void updateRGBLED(int lightLevel) {
  int redValue = 0, greenValue = 0, blueValue = 0;

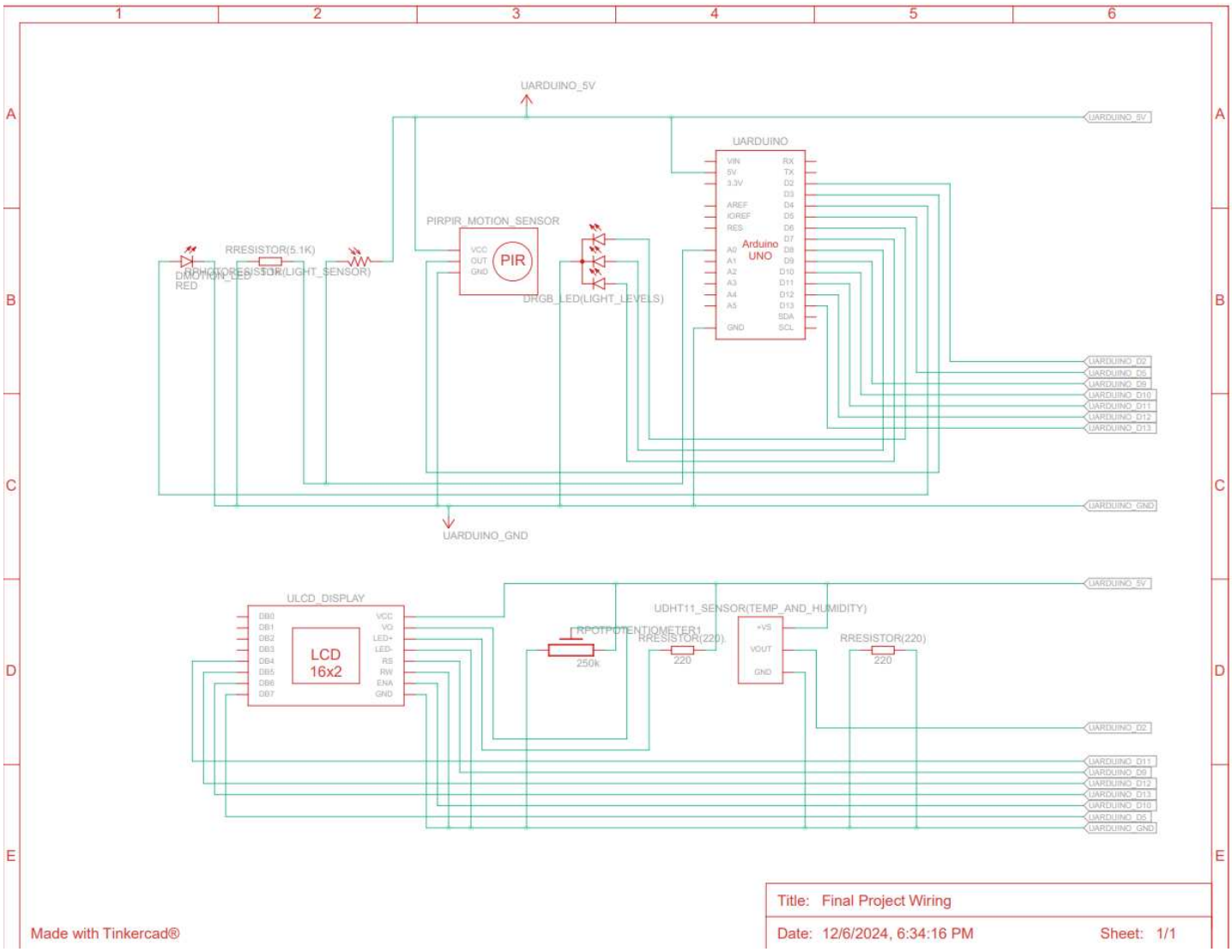
  if (lightLevel <= 39) {
    // Red for 0-39%
    redValue = 255;
    greenValue = 0;
    blueValue = 0;
  } else if (lightLevel >= 40 && lightLevel <= 60) {
    // Blue for 40-60%
    redValue = 0;
    greenValue = 0;
    blueValue = 255;
  } else if (lightLevel >= 61) {
    // Green for 61-100%
    redValue = 0;
    greenValue = 255;
    blueValue = 0;
  }

  // Write the RGB values to the corresponding pins
  analogWrite(RGB_RED_PIN, redValue);
  analogWrite(RGB_GREEN_PIN, greenValue);
  analogWrite(RGB_BLUE_PIN, blueValue);
}

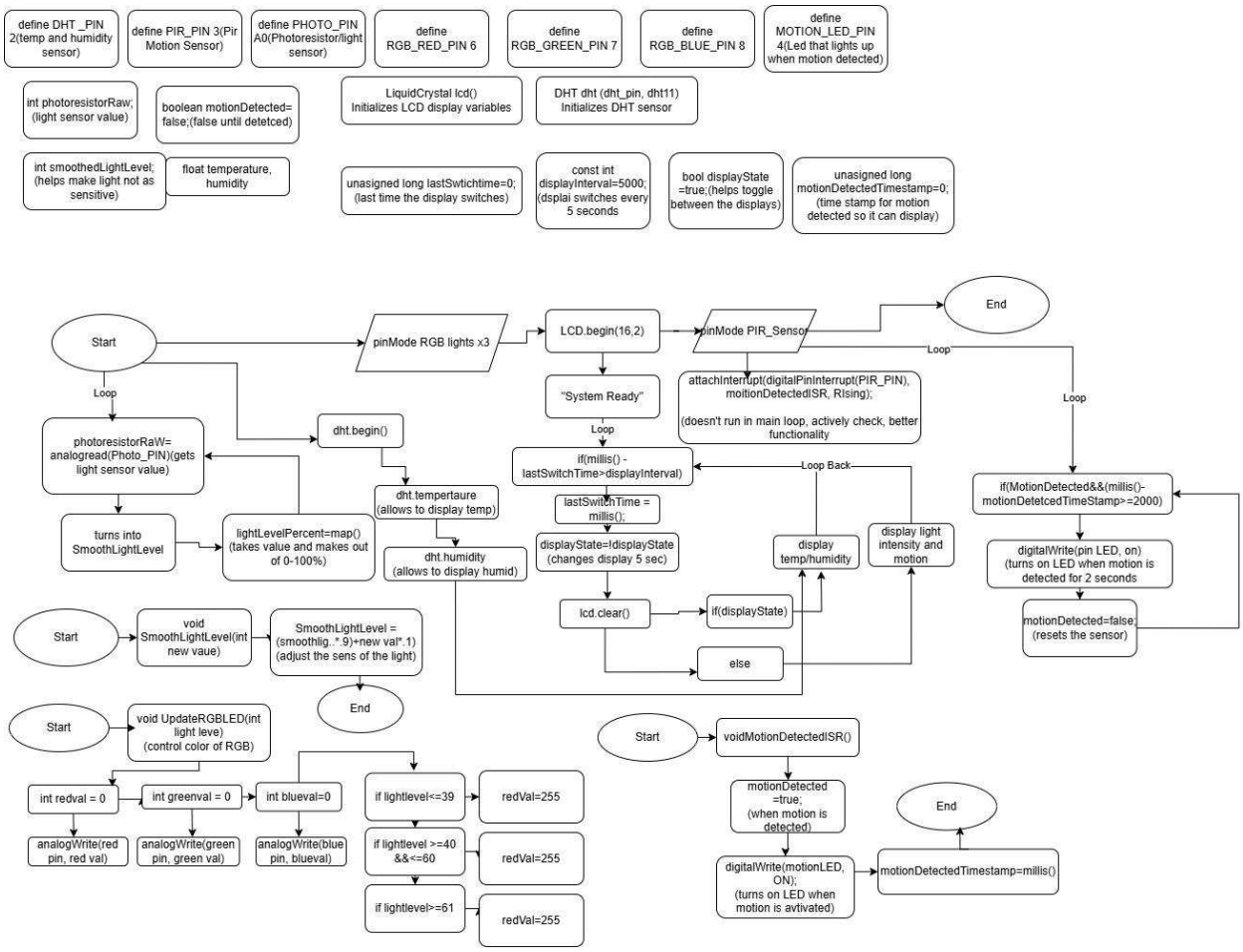
// Interrupt Service Routine (ISR) for PIR motion sensor
void motionDetectedISR() {
  motionDetected = true;  // Set the motionDetected flag to true when motion is detected
  digitalWrite(MOTION_LED_PIN, HIGH);  // Turn on the motion LED immediately when motion is detected
  motionDetectedTimestamp = millis();  // Record the timestamp when motion was detected
}
```

Append schematics, functional diagrams and support text for your project here;

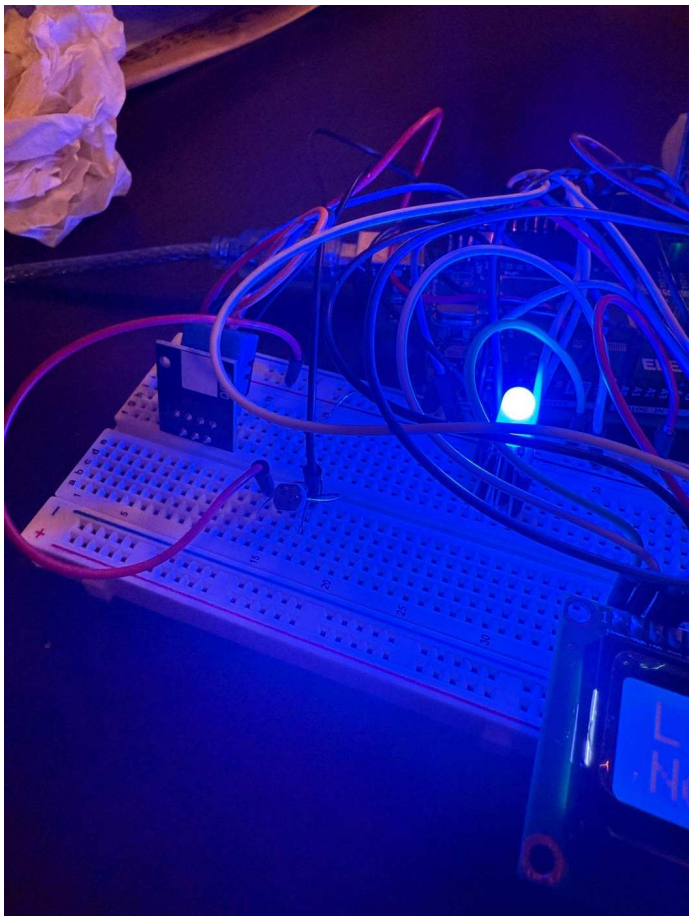
This is my Wiring Diagram/Flow Chart for the project



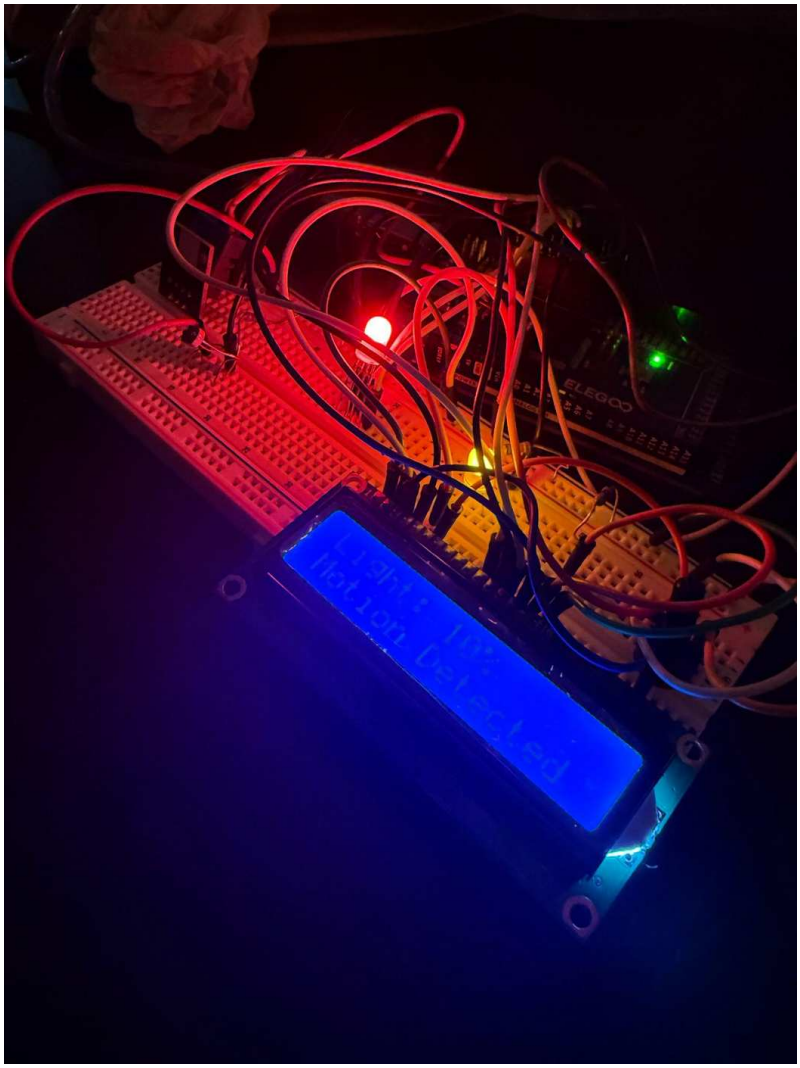
This is my Software Diagram/Flow Chart for the project(pdf file attached on submission)



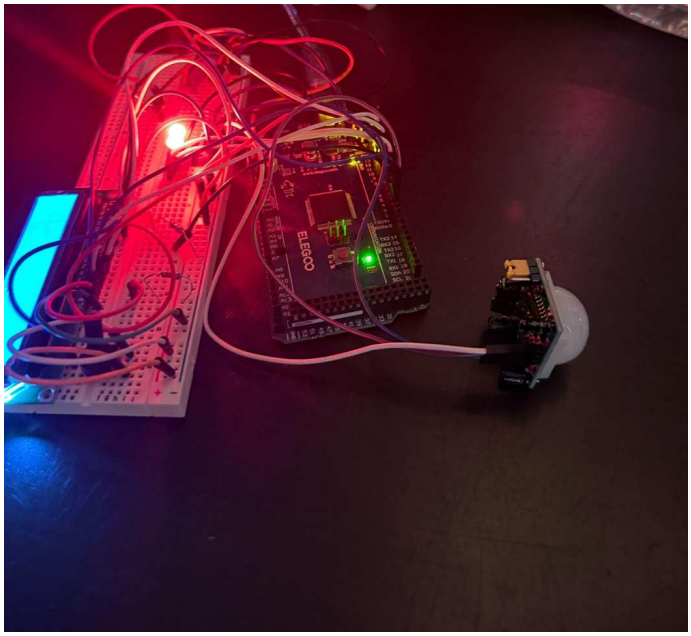
Append pictures of your project here with explanations;



This picture shows a photoresistor(light sensor) that senses light in the environment and changes the light accordingly. As you can see the light level in this room is medium so the light is blue. You can also see the DHT11 sensor in the top left that reads temperature and humidity.



In this picture, the light level is low so the RGB turns red. Motion is also detected so it displays on LCD, which was hard to show since it doesn't pick up well on the camera, and the yellow LED turns out for two seconds.



‘

Another view of my project and how the PIR motion sensor is attached to my project, the white ball above is what detected the motion and reads the value when motion is detected.

Append a link to a YouTube video describing your project (link also in the comments of submission) here;https://youtu.be/l4RWbLrMwmI